

Appendix A — Glossary & Jargon Index

Every term footnoted across the book, defined once
and pointed back to the chapter that treats it in full.

SUBJECT hathor-core · Appendix A

CHAPTER A · Appendices

BRANCH study/hathor-core-studies

EDITION 2026 · v1

Glossary · Jargon index · 298 terms · Cross-referenced to chapters

Table of Contents

Appendix A — Glossary & Jargon Index	3
A	3
B	4
C	6
D	9
E	12
F	12
G	14
H	15
I	16
J	17
K	17
L	18
M	19
N	20
O	21
P	21
R	25
S	26
T	29
U	31
V	31
W	32
X	32

Appendix A — Glossary & Jargon Index

This appendix collects every term the book defines in a footnote — 298 of them — into one alphabetical reference. Each entry gives a one- or two-sentence definition and a pointer to the chapter (or chapters) where the term is introduced and used in context. Where a term recurs across several chapters, all are listed; the chapter in **bold-ish first position** is usually where it is first defined.

Use this the way a junior reader should: when a later chapter uses a word you have forgotten, look it up here for the quick definition, then follow the chapter pointer if you need the full treatment. Terms are sorted ignoring case and punctuation, so `base58check`, `BFS`, and `BIP32` sit where you would expect them.

How to read an entry. **Headword** · Ch N — definition. The chapter number is a clickable pointer in the source and a lookup target in print.

A

Abstract base class · Ch 1, Ch 25, Ch 26, Ch 27 — An abstract base class (ABC) is a class you cannot instantiate directly; it declares methods (often `@abstractmethod`) that concrete subclasses must implement. `Serializer` and `Deserializer` are ABCs, with `BytesSerializer` / `BytesDeserializer` as the concrete in-memory implementations. Full treatment in Chapter 1.

Abstraction · Ch 1 — Abstraction = exposing what an operation does while hiding how it is done, so callers depend on a simple contract rather than complex internals.

Account model · Ch 7 — The account model (or account/balance model) records ownership as a mutable balance per account; a transaction debits one balance and credits another. Used by banks and by Ethereum. Contrast the UTXO model.

Acyclic · Ch 8 — Acyclic means containing no cycles — no path that follows edges and returns to its starting node. A directed acyclic graph (DAG) combines directedness with acyclicity.

Adapter · Ch 3 — An adapter wraps an object to present a different interface than it natively offers, so it can be used where that other interface is expected. Also called a wrapper.

Any · Ch 5 — `Any` is the type that is compatible with everything and disables checking for the values it touches. It is the deliberate escape hatch from the type system; overusing it defeats the purpose of typing.

Argparse · Ch 19 — `argparse` is Python's standard-library module for parsing command-line arguments: you declare the options a program accepts, and it parses `sys.argv` into a structured object and generates help text. It handles only the command line.

Args in a function signature collects any extra** · *Ch 4* — args in a function signature collects any extra **positional** arguments into a tuple. In a call, `seq` unpacks a sequence into separate positional arguments. The name `args` is convention; the `*` is what matters.

Assertnever · Ch 31 — `assert_never(x)` is a typing helper that the static type-checker treats as "this line must be unreachable." If the checker can prove some case reaches it (e.g. an unhandled enum value), the build fails. At runtime it raises if ever actually hit. It is how a `match` over a fixed set of cases is made provably exhaustive.

Asymmetric cryptography · Ch 40 — **Asymmetric cryptography** (also "public-key cryptography") uses a pair of mathematically linked keys — one private, one public — instead of a single shared secret. What one key does (e.g. sign), only the other can verify; you cannot derive the private key from the public key.

Attribute · Ch 1 — An attribute is a named piece of data attached to an object (e.g. `account.balance`) or to a class. Reached with the dot operator.

Authority output · Ch 25 — An authority output carries no spendable value; instead its `value` field is reinterpreted as a permissions bitmask (mint and/or melt rights for a token). It is flagged by the high bit of the output's `token_data` byte. Authorities are how a token issuer retains control over a token's supply — see Chapter 7 §7.6.

Autobahn · Ch 36 — **Autobahn** is a Python library implementing the WebSocket protocol (and the higher-level WAMP protocol, unused here). `hathor-core` uses its Twisted binding, `autobahn.twisted.websocket`, whose `WebSocketServerFactory` / `WebSocketServerProtocol` mirror Twisted's own Factory/Protocol pair.

Autocomplete · Ch 5 — Autocomplete is the editor feature that suggests valid attributes/methods as you type. Type information makes these suggestions accurate, because the editor knows each variable's type.

B

Back-pressure · Ch 30 — Back-pressure is a flow-control technique where a fast producer is held back so a slower consumer is not overwhelmed. Here the client declares a window (how many unacknowledged events it can hold), and the node refuses to send beyond it until the client acknowledges, keeping memory bounded on both sides.

Base58check · Ch 40 — **base58check** is an encoding for addresses: a version byte, then the payload, then a 4-byte checksum (`sha256(sha256(...))` truncated), all encoded in base58 (digits/letters minus look-alikes like 0/O, I/l). The checksum makes mistyped addresses detectable.

Base image · Ch 15 — A base image is the existing image a Dockerfile starts `FROM` (here an official Python-on-Debian image). You build on top of it rather than from an empty filesystem.

BFS · Ch 32 — **BFS** (breadth-first search) — a graph-traversal strategy that visits a starting node, then all of its immediate neighbors, then their neighbors, and so on, in expanding "rings." Hathor's `BFSTimestampWalk` visits descendants in timestamp order and is the engine behind voiding propagation.

BIP32 · Ch 40 — **BIP32** is the standard defining hierarchical-deterministic key derivation: a function that grows a tree of child keys from one master key, so a single master secret regenerates every key. ("BIP" = Bitcoin Improvement Proposal.)

BIP39 · Ch 40 — **BIP39** is the standard mapping a list of memorable words (the mnemonic) to the binary seed that BIP32 starts from. It is why a wallet backup can be 24 words on paper instead of raw bytes.

BIP9 · Ch 38 — **BIP9** ("Bitcoin Improvement Proposal 9") is the Bitcoin scheme for "versionbits" soft-fork activation: miners signal readiness for a change by setting bits in a block's version field, and the change activates once a threshold of blocks in a retarget window signal support, within a defined timeout. Hathor's feature activation is a close adaptation of it, with the addition of the `MUST_SIGNAL` mandatory-signalling phase.

Block · Ch 6 — A block is a batch of transactions grouped and added to the ledger together. In classic blockchains, blocks are the units that form the chain; in Hathor, blocks coexist with free-standing transactions in a graph (Ch 8).

Blocking · Ch 2, Ch 16 — A blocking call does not return control to your program until it has finished; while it waits, the calling thread can do nothing else. Its opposite is non-blocking (returns immediately, result delivered later). Full treatment in Ch. 2.

Bloom filter · Ch 27 — A **Bloom filter** is a compact, probabilistic membership test: it can answer "definitely not present" with certainty, or "possibly present" (with a small false-positive rate). RocksDB keeps one per data file so a lookup can skip files that certainly do not hold the key. `key_may_exist` exposes this — a fast "no" without touching the data, which is why `transaction_exists` probes it before doing a real read.

Blueprint · Ch 39 — A Blueprint is Hathor's term for a contract template — a Python class defining the contract's state and methods. One Blueprint can back many independent contracts, just as one class can have many object instances.

Boundary · Ch 18 — A boundary is any point where data enters a program from outside (files, network, user input, other systems). Data at the boundary is untyped and untrusted and must be validated before the program relies on it.

Build backend · Ch 13 — A build backend is the component that turns a project's source into installable distributions (a wheel and/or sdist). `pyproject.toml`'s `[build-system]` table names it; Poetry provides its own.

Builder · Ch 0, Ch 3 — A builder is a design pattern: an object whose job is to construct another, complicated object step by step. Hathor's builder constructs the fully-wired node. The pattern is explained in Chapter 3; Hathor's builders in Chapter 24.

Build system · Ch 14 — A build system turns source into artifacts, tracking dependencies and timestamps to rebuild only what changed. Make is one; for compiled languages this incremental rebuild is its main value, mostly unused in Python projects.

Byte · Ch 26 — A byte is eight bits, holding an integer from 0 to 255. All files and network data are sequences of bytes; everything else — text, numbers, objects — is an interpretation layered on top of bytes by agreement.

Bytecode · Ch 11, Ch 39 — Bytecode is a low-level, compact instruction format that a program is compiled into and that a virtual machine executes, rather than running the original source directly. Ethereum stores contract bytecode on-chain; Hathor stores Python source and runs it on the interpreter.

C

Callback · Ch 2 — A callback is a function passed to other code to be called ("called back") when a specific event happens or a result becomes available.

Cast · Ch 23 — `typing.cast(T, x)` tells the type checker "treat `x` as type `T`." It performs no runtime check or conversion; it only changes how the checker reasons about the value — which is why Hathor pairs it with a runtime `verifyObject` check.

Change · Ch 7 — Change is an output a transaction sends back to the payer, equal to the consumed inputs minus the amount paid (minus any fee). It exists because outputs must be spent whole, like receiving change after paying cash with a larger note.

Checkpoint · Ch 6, Ch 10, Ch 29, Ch 32 — **Checkpoint** — a hard-coded `(height, hash)` pair in the node's settings marking a block the network agrees is final. The node refuses any reorg that would rewrite history before a checkpoint, putting a hard floor under otherwise-probabilistic finality. Full treatment in Ch. 10.

Chokepoint · Ch 33 — A chokepoint (or single point of control) is a deliberate design where all instances of some operation are routed through one piece of code, so the logic exists in exactly one place. The trade-off: it can become a bottleneck or a single point of failure, but it guarantees consistency. Here, consistency is worth far more than the negligible overhead.

Classmethod · Ch 4 — `@classmethod` marks a method whose first argument is the class itself (`cls`) rather than an instance. Common for alternative constructors (e.g. `Vertex.from_bytes(...)`).

Closure · Ch 4 — A closure is a function that retains access to variables from the scope in which it was defined, even after that scope has finished executing. The inner function "closes over" those variables.

Code coverage · Ch 20 — Code coverage measures which lines (or branches) of the code are executed by the test suite, reported as a percentage. High coverage doesn't guarantee good tests, but low coverage reliably reveals untested code.

Column family · Ch 27 — A **column family** is a named, independent key-space inside a single RocksDB database — like a separate dictionary that shares the same database file and write-ahead log but keeps its own keys and can be tuned and compacted on its own. Hathor uses five: `tx`, `meta`, `static-meta`, `attr`, `migrations`.

Compaction · Ch 27 — **Compaction** is the LSM-tree's background housekeeping: it merges several sorted on-disk files into fewer, dropping values that have been overwritten or deleted, so reads stay fast and disk space is reclaimed. It runs even when the node is otherwise idle.

Composition · Ch 1 — Composition builds an object by giving it other objects as attributes and delegating work to them — a "has-a" relationship, as opposed to inheritance's "is-a."

Composition root · Ch 24 — A **composition root** is the single place in an application where the

Concurrency · Ch 2 — Concurrency is making progress on multiple tasks over the same period by interleaving them. It is about structure (dealing with many things at once), not necessarily simultaneity.

Configargparse · Ch 19 — `configargparse` is a Python library that subclasses the standard `argparse`, letting each declared option be set from a command-line flag, an environment variable, or a config file, with a defined precedence — a drop-in upgrade for `argparse`-based programs.

Confirmations · Ch 10 — Confirmations are the confirming vertices (especially blocks) accumulated on top of a transaction; more confirmations mean more work an attacker must overcome, so recipients wait for a threshold before trusting a payment.

Confirmed · Ch 8 — A vertex is confirmed by every later vertex that can reach it via parent edges. More confirmations (and accumulated work behind them) make a vertex harder to reverse — the DAG analogue of chain depth.

Conflict · Ch 10 — A conflict is a pair (or set) of individually-valid transactions that cannot all be part of the ledger — classically, two transactions spending the same output (a double-spend). Distinct from an invalid transaction, which breaks a rule on its own.

Console-script entry point · Ch 21 — A console-script entry point is a line in packaging metadata of the form `name = "package.module:function"`. At install time the packaging tool generates an executable called `name` on your `PATH` that imports and calls that function. It is how a `pip install` makes a shell command appear.

Constructor · Ch 1 — A constructor is the routine that creates and sets up a new object. In Python the setup step is `__init__`; technically object creation is handled by `__new__`, which you almost never need to touch.

Container · Ch 15 — A container is a running instance of an image: an isolated process (or process tree) with its own filesystem view, using the image's contents. Many containers can run from one image.

Containerization · Ch 15 — Containerization is packaging an application with everything it needs to run into a container image, so it runs consistently across different machines regardless of their host configuration.

Content-addressed · Ch 29 — Content-addressed means a piece of data is identified by the hash of its own bytes. Change one byte and the hash changes, so a vertex cannot be silently altered without its id changing — which is why the blocks and transactions themselves cannot be quietly corrupted, only the separately-stored metadata about them.

Context manager · Ch 33 — A context manager is the `with ...:` construct. `non_critical_code(self._log)` wraps a block so that an exception inside it is logged and swallowed rather than propagated — the boundary between "fatal if it fails" and "tolerable if it fails."

Continuous integration · Ch 5, Ch 20 — Continuous integration (CI) is an automated system that runs checks (linters, type checker, tests) on every proposed change and blocks merging unless they pass — the gate that makes the toolchain mandatory.

Cooperative · Ch 2 — Cooperative multitasking relies on each task voluntarily yielding control (e.g. at an `await`) so others can run. No task is forcibly interrupted, so a task that never yields blocks all others.

Coroutine · Ch 2 — A coroutine is a function that can suspend its execution (at an `await`) and later resume from exactly that point, preserving its local state. Defined with `async def` in Python.

CPU-bound · Ch 2 — CPU-bound describes work whose speed is limited by raw computation — the processor is the bottleneck. Examples: hashing, encryption, large numeric loops.

CPython · Ch 11 — CPython is the reference implementation of the Python interpreter, written in C. It is what you get from python.org and what runs by default when you type `python`. Alternatives (PyPy, Jython, GraalPy) exist but are not used by `hathor-core`.

Cryptographic hash function · Ch 6 — A cryptographic hash function maps input of any size to a fixed-size output (the hash), with properties — determinism, one-wayness, avalanche, collision-resistance — that make it usable as a tamper-evident fingerprint. Examples: SHA-256, SHA-3.

Curses · Ch 42 — curses is a standard library for building text-based, full-screen terminal user interfaces (menus, live-updating tables) instead of plain scrolling output. `hathor-cli top` uses it to draw a live `top`-style dashboard.

D

DAA · Ch 9, Ch 31, Ch 37 — **DAA** = Difficulty Adjustment Algorithm. The code that decides how hard mining should be right now, so blocks keep arriving at a steady average rate as total network hashing power rises and falls. Full treatment in Ch 32 / see Ch 9.

Daemon · Ch 0, Ch 41 — A daemon is a program that runs continuously in the background rather than finishing and exiting. Web servers and databases are daemons. The word is pronounced "demon" and comes from an old Unix tradition.

DAG · Ch 0, Ch 35 — **DAG** = Directed Acyclic Graph. A graph is a set of items ("nodes") joined by links ("edges"). Directed means each edge points one way. Acyclic means you can never follow the edges in a loop back to where you started. Full treatment in Chapter 8.

Deadlock · Ch 2 — A deadlock is a standstill where two or more threads each wait for a resource the other holds, so none can ever proceed.

Decorator · Ch 4, Ch 42 — A decorator is a callable that takes a function (or class) and returns a replacement, applied with `@name` above a definition. It is the standard Python way to wrap behavior around a function.

Decoupling · Ch 3, Ch 30 — Decoupling means reducing how much two pieces of code need to know about each other. Tightly coupled code shares references and assumptions, so a change to one forces changes to the other. Loosely coupled code communicates through a narrow, stable interface — here, "publish an event" — so each side can change freely.

Deferred · Ch 29, Ch 34, Ch 35 — A Deferred is Twisted's placeholder for a result that is not ready yet — a "future." You attach callbacks (run on success) and errbacks (run on failure); the reactor fires them when the result arrives. Full treatment in Ch. 16.

Dependency hell · Ch 12 — Dependency hell is the situation where multiple projects (or libraries) require conflicting versions of the same package, making it impossible to satisfy them all from a single shared installation.

Dependency injection · Ch 3 — Dependency injection is the practice of passing an object its collaborators from outside (e.g. via its constructor) rather than having it create or fetch them itself. It makes dependencies explicit and substitutable (e.g. for testing).

Dependency resolution · Ch 13 — Dependency resolution is computing a single set of package versions that simultaneously satisfies every direct and transitive constraint. It can be computationally hard and is the core job of a dependency manager.

Derived state · Ch 28 — Derived state (also secondary state) is data computed from some primary source of truth, holding nothing the primary does not already imply. Its defining property is that it can be discarded and recomputed. Hathor's indexes are derived from the vertices in storage; storage is primary. This is what makes index corruption recoverable by rebuild rather than fatal.

Descriptor · Ch 39 — A descriptor is a Python object that customizes what happens when an attribute is read, written, or deleted, by defining `__get__` / `__set__` / `__delete__`. Properties are descriptors. Hathor's `Field` is a descriptor that routes `self.count` to the trie instead of to memory.

Design pattern · Ch 3 — A design pattern is a named, reusable solution to a commonly occurring problem in software design — a description of a shape to apply, not a finished piece of code.

Deterministic · Ch 39 — Deterministic execution means the same inputs always produce exactly the same outputs on every machine. For consensus this is mandatory: if two nodes computed different results for the same contract call, they would disagree on the ledger. It rules out wall-clock time, true randomness, and reading external data.

Digest · Ch 6 — A digest (or hash) is the fixed-size output of a hash function — here, 256 bits, usually written as 64 hexadecimal characters. It acts as a compact fingerprint of the input.

Digital signature · Ch 7, Ch 40 — A digital signature is data produced with a private key that anyone can verify with the matching public key, proving the signer authorized a specific message without revealing the private key. It is how a locking script confirms a spender's authority. (Full treatment with the crypto libraries in Ch 40.)

Directed graph · Ch 8 — A directed graph has edges with a direction — each edge goes from one node to another, not symmetrically. An edge $A \rightarrow B$ does not imply $B \rightarrow A$.

Dispatch · Ch 4, Ch 26 — Dispatch means selecting which piece of code to run based on a value at runtime — here, choosing a vertex class based on the version byte. Full treatment of dispatch as a pattern is in Chapter 4.

Dispatch table · Ch 4 — A dispatch table is a data structure (usually a dict) mapping keys — message types, command names, event names — to the functions that handle them, replacing a long `if/elif` chain.

Distributed ledger · Ch 6 — A distributed ledger is a transaction record replicated across many independent computers, each holding a full copy, with no central master. Agreement among the copies replaces trust in a single operator.

Docker · Ch 15 — Docker is a platform for building and running containers — isolated, portable packages of an application together with its entire runtime environment (libraries, interpreter, OS userland).

Dockerfile · Ch 15 — A Dockerfile is a text file of instructions (`FROM`, `RUN`, `COPY`, `ENTRYPOINT`, ...) describing how to build an image. Each instruction produces a cached layer.

Domain-specific language · Ch 43 — A domain-specific language (DSL) is a small, purpose-built notation for one narrow job — here, describing a DAG of blocks and transactions — as opposed to a general-purpose language like Python. The `dag_builder` DSL is parsed by a hand-written tokenizer; it is not Python, though a few attribute values are parsed as Python expressions.

Double-spend · Ch 6, Ch 32 — **Double-spend** — two different transactions that both try to spend the same output. Each is valid alone; together they conflict, and consensus must void at least one so a coin is never spent twice. The visible "collision" is exactly what the spent-once rule and `voided_by` machinery exist to catch.

Duck typing · Ch 1 — Duck typing is Python's style of polymorphism: an object's suitability is determined by whether it has the needed methods/attributes, not by what class it inherits from. "If it quacks, it's a duck."

Dunder · Ch 1 — Dunder = "double underscore." Dunder methods like `__init__`, `__repr__`, `__eq__`, `__len__` let your objects integrate with Python's built-in syntax and functions. Also called magic or special methods.

Dynamic dispatch · Ch 4 — Dynamic dispatch selects the implementation at run time based on an object's actual type — the mechanism behind method calls and polymorphism. Contrast static dispatch, decided at compile time by the declared type.

Dynamic typing · Ch 5 — Dynamic typing means variable types are not fixed and are checked while the program runs. A name can be bound to a value of any type, and type errors surface at execution time. Python, Ruby, and JavaScript are dynamically typed.

E

ECDSA · Ch 40 — **ECDSA** = Elliptic Curve Digital Signature Algorithm. The specific signature scheme Hathor (and Bitcoin) use. "Elliptic curve" is the kind of one-way maths that makes private→public easy and the reverse infeasible. You can read this chapter without the maths.

Encapsulation · Ch 1 — Encapsulation = bundling data with the methods that operate on it, and restricting outside access to that data so it can only change through approved methods.

Endianness · Ch 26 — Endianness is the order in which the bytes of a multi-byte number are laid down. Big-endian puts the most-significant byte first (the way we write decimal); little-endian puts the least-significant byte first. Neither is more correct; writer and reader need only agree. Hathor uses big-endian everywhere.

Environment variable · Ch 19, Ch 22 — An environment variable is a named value the operating system makes available to a running process (read in Python via `os.environ`). It is a process-global channel: any part of the program can read it without being passed it explicitly, which is why it suits a setting that must be reachable from hundreds of call sites.

Event loop · Ch 0 — An event loop is a program structure that repeatedly waits for "something to happen" and then dispatches a handler for it, instead of running top-to-bottom and blocking. It is the core idea behind asynchronous programming — Chapter 2.

Event queue · Ch 30 — The event queue is Hathor's name for the durable, numbered log of events on disk. "Queue" here means an ordered sequence consumers read in order, not a transient in-memory buffer. It is the persistent half of the event system, off by default and enabled per node.

EVM · Ch 39 — The EVM (Ethereum Virtual Machine) is Ethereum's purpose-built virtual machine: a stack-based interpreter with its own instruction set that every Ethereum node runs to execute contract bytecode. Hathor has no equivalent separate machine — contracts run on the node's own (sandboxed) Python interpreter.

Exit code · Ch 21 — An exit code (or status code) is the small integer a process returns to the shell when it ends. `0` means success; any non-zero value signals an error. Shells and deployment tools branch on it (`if program; then ...`). Hathor uses `0` for success, `1` for a `Ctrl-C` abort, `2` for an uncaught crash.

F

Facade · Ch 3 — A facade is an object that provides a simplified, high-level interface to a larger, more complex subsystem, hiding its coordination behind a few easy methods.

Factory · Ch 3, Ch 34, Ch 35, Ch 37 — In Twisted, a factory is the object that creates a fresh protocol instance for each incoming connection and holds state shared across all of them. The `StratumFactory` builds one `StratumProtocol` per miner and holds the list of all miners. Full treatment of factories in Chapter 16 (and the pattern in Chapter 3).

Fail-fast · Ch 18 — Fail-fast is the principle of detecting and reporting an error as early and as close to its source as possible — here, validating input at the boundary so a bad value raises immediately with a clear message rather than causing an obscure failure later.

Failure · Ch 16 — A Twisted Failure is an object that captures an exception together with its traceback, so an error can be carried along a Deferred's errback chain (and inspected later) rather than being raised and lost. Think of it as "an exception, boxed for asynchronous travel."

Fallback · Ch 4 — A fallback is an alternative path taken when the preferred one is unavailable or fails — a default value, a backup service, or a simpler implementation. It makes code degrade gracefully instead of breaking.

Feature · Ch 33 — A feature here is a protocol upgrade gated by feature activation — a mechanism where rule changes switch on according to a schedule and miner signalling. `VerificationParams` carries which features are active so verification applies the right rules. Ch. 38.

Feature activation · Ch 22 — Feature activation is Hathor's mechanism for switching protocol upgrades on over a schedule, by miner signalling. The schedule for a network is part of its settings profile (the `FEATURE_ACTIVATION` field). Full treatment in Ch. 38.

Finality · Ch 6, Ch 10 — Finality is the assurance that a transaction cannot be reversed. In proof-of-work systems it is probabilistic: never absolute, but exponentially more certain as more blocks are built on top (greater depth).

First-class · Ch 2 — A value is first-class if it can be stored in variables, passed as an argument, and returned from functions. In Python, functions are first-class values.

Flake8 · Ch 20 — flake8 is a Python linter that combines several checkers (pyflakes for likely bugs, pycodestyle for style) to report problems by static inspection. Configurable line length and per-file ignores.

Fluent interface · Ch 3 — A fluent interface is an API style where methods return the object itself so calls can be chained (`obj.a().b().c()`), reading almost like a sentence.

Fork · Ch 6, Ch 38 — A fork (specifically a consensus fork or chain split) is when nodes stop agreeing on which blocks are valid, so the single shared ledger splits into two incompatible histories. A coin's state can differ between the two sides. Feature activation exists to prevent forks caused by rule changes; this is distinct from a temporary fork, the brief, self-healing disagreement when two miners find a block at nearly the same time (resolved by consensus, Ch. 32).

Formatter · Ch 20 — A formatter automatically rewrites code into a canonical style (spacing, import order) so formatting is consistent and never argued over. `isort` formats imports; it changes files rather than only reporting.

Frame · Ch 36 — A **frame** is the unit of data in the WebSocket protocol: a small message with a tiny header indicating its type (text, binary, ping, pong, close) and length. After the handshake, the two sides exchange frames instead of HTTP requests/responses.

Fuel · Ch 39 — Fuel is Hathor's name for the metered execution budget — the equivalent of Ethereum's gas. The design charges fuel per executed bytecode operation and aborts when it reaches zero; this branch threads the budget through but does not yet decrement it (see §39.A.7).

Full-duplex · Ch 36 — **Full-duplex** describes a channel on which both ends can transmit at the same time and either can start at any moment (like a phone call). **Half-duplex** allows only one direction at a time (like a walkie-talkie). Plain HTTP is effectively half-duplex and always client-initiated; a WebSocket is full-duplex.

Future · Ch 2 — A future (a.k.a. promise) is an object representing a result that will be available later. You attach callbacks to it or `await` it. Twisted's version is the Deferred; JavaScript's is the Promise.

G

"Gang of Four" · Ch 3 — The "Gang of Four" (GoF) are the four authors of Design Patterns: Elements of Reusable Object-Oriented Software (1994), the book that catalogued and named 23 classic patterns and made the vocabulary standard.

Gas · Ch 39 — Gas is Ethereum's unit of computational cost: each operation costs a fixed amount of gas, the transaction carries a gas budget, and execution aborts if the budget runs out. It is the mechanism that makes unbounded loops harmless. Hathor's equivalent budget is called fuel.

Generic · Ch 5, Ch 25 — A generic class is parameterized by a type variable, written `Class[T]`. `GenericVertex[StaticMetadataT]` is generic over its static-metadata type, so `Block = GenericVertex[BlockStaticMetadata]` and `Transaction = GenericVertex[TransactionStaticMetadata]` stay distinct to the type-checker. Full treatment in Chapter 5.

Genesis · Ch 0, Ch 22, Ch 29, Ch 35 — The genesis is the hard-coded first block and initial transactions that every node on a network agrees on by definition — the shared root of history. Different networks have different genesis data; that difference is part of what makes them distinct networks. Full treatment in Ch. 25 & 32.

Geometric distribution · Ch 43 — The geometric distribution models the number of independent yes/no trials until the first success, when each trial succeeds with probability p . Mining is exactly that — repeated nonce attempts, each a long shot — so drawing one geometric sample reproduces "how long mining would have taken" without running the trials.

Global Interpreter Lock · Ch 2, Ch 16 — The Global Interpreter Lock is a mutex in standard CPython that allows only one thread to execute Python bytecode at a time. It simplifies the interpreter but prevents threads from running Python computation in parallel — a reason the ecosystem leans on the single-threaded event-loop model. Full treatment in Ch. 2.

Global state · Ch 3 — Global state is data accessible from anywhere in the program. It makes code harder to reason about and test, because any function might read or change it invisibly.

Gradual typing · Ch 5 — Gradual typing allows mixing typed and untyped code in the same program, so annotations can be added incrementally rather than all at once. Python's type system is gradual.

Graph · Ch 8 — A graph is a set of nodes connected by edges. It is an abstract structure for representing relationships; many systems (networks, maps, dependencies, ledgers) are naturally graphs.

H

Halting problem · Ch 39 — The halting problem is the proven impossibility of writing a general algorithm that decides, for any program and input, whether it will eventually stop or run forever. Its consequence here: the node cannot prove a contract terminates, so it must instead bound execution with a finite budget.

Hash160 · Ch 40 — **hash160** is SHA-256 followed by RIPEMD-160, producing a 20-byte fingerprint of a public key. Hashing the public key shortens the address and hides the public key until the output is spent.

Hash function · Ch 40 — A **hash function** maps a message of any size to a fixed-size fingerprint, such that any change to the message changes the fingerprint unpredictably and you cannot run it backwards. Hathor uses SHA-256 and RIPEMD-160. Signatures are computed over a hash of the message, not the raw message.

Hash pointer · Ch 6 — A hash pointer is a reference to a piece of data that includes that data's hash, so the reference both locates the data and certifies it is unchanged. Chaining blocks by hash pointer is what makes the history tamper-evident.

Header · Ch 39 — A header here is a self-describing block of extra bytes appended to a transaction's serialized form, carrying optional structured data. The nano-header carries the instructions for a contract call; a separate fee header carries fee data.

Heap · Ch 43 — A heap is a tree-shaped data structure kept partly ordered so that the smallest (or largest) item is always at the front and cheap to remove. Python's `heapq` gives this over a plain list. The simulator uses a min-heap keyed by fire-time so "the next timer to fire" is always at hand.

Higher-order function · Ch 2 — A higher-order function is one that takes other functions as arguments and/or returns a function. `map` , `sorted(key=...)` , and decorators are examples.

I

Idempotent · Ch 33 — Idempotent means running an operation more than once has the same effect as running it once. The handler's gate is idempotent in spirit: re-ingesting an already-accepted vertex is detected as "already-known" and does nothing, so a vertex can safely arrive at the gate repeatedly.

Image · Ch 15 — An image is a read-only, layered package of a filesystem (OS userland, libraries, app) — the blueprint from which containers are created. Built from a Dockerfile, stored in registries.

Immutable · Ch 1 — Immutable data cannot be changed after creation; you make a modified copy instead. Python strings and tuples are immutable; lists and dicts are mutable (changeable in place).

Index · Ch 28 — An **index**, in databases, is a derived (secondary) data structure that precomputes the answer to a class of lookup queries, keyed the way the query is phrased, so the answer can be found without scanning all the primary data. It costs extra storage and extra work on every write, in exchange for fast reads. The index at the back of a book — topic → page numbers — is the same idea on paper. Because it is computed from the primary data, it can always be rebuilt and contains no information that is not already implied by that data.

Inheritance · Ch 1 — Inheritance lets a subclass acquire the attributes and methods of a superclass, then add or override behavior. Models an "is-a" relationship.

Inlinecallbacks · Ch 35 — `@inlineCallbacks` is a Twisted decorator that lets you write asynchronous code as if it were straight-line: `yield some_deferred` pauses the function until the result arrives, instead of nesting callbacks. Ch. 16.

Input · Ch 7 — An input of a transaction is a reference to a specific earlier output that this transaction consumes (spends). In Hathor it is the `(tx_id, index)` pair plus an unlocking script. An input does not hold value itself; it points at the output that does.

Instance · Ch 1 — An instance is one concrete object built from a class. "Instantiate" means "create an instance." `Account("Alice")` instantiates the `Account` class.

Interface · Ch 23 — An interface is a named list of methods an object promises to provide — a contract. It says nothing about implementation, only about the shape of the object.

Interfaces · Ch 16, Ch 23 — `zope.interface` is a library that lets Python code declare interfaces — named sets of methods a class promises to provide — and verify at runtime that an object implements one. Twisted uses it to describe the reactor's capabilities (`IReactorCore`, `IReactorTime`, `IReactorTCP`). It is an older, runtime-checked cousin of `typing.Protocol`.

Intersection type · Ch 23 — An intersection type describes a value that satisfies several contracts at once. Python has no dedicated syntax for it, so the idiom is to define a type that inherits from each contract, as `ReactorProtocol` does with its three slice-protocols.

Invariant · Ch 1 — An invariant is a condition that must always hold true for an object to be valid — e.g. "an account's balance is never negative." Much of good design is about identifying invariants and ensuring no code can violate them.

I/O-bound · Ch 2, Ch 16 — I/O-bound describes work whose speed is limited by input/output — network, disk, waiting on other systems — rather than by the processor. A node is overwhelmingly I/O-bound. Full treatment in Ch. 2.

Isort · Ch 20 — isort is a tool that automatically sorts and groups Python import statements into a consistent order (standard library, third-party, first-party). Run in "fix" mode it edits files; in "check" mode it only verifies.

J

JSON · Ch 17 — JSON (JavaScript Object Notation) is a text format for structured data as key-value objects and arrays. Emitting logs as one JSON object per line makes them directly ingestible by log-analysis tools.

JSON-RPC 2.0 · Ch 37 — JSON-RPC 2.0 is a tiny standard for remote procedure calls encoded as JSON: each message is an object with a `method`, `params`, and an `id` (for requests), or a `result / error` (for responses). Stratum is JSON-RPC sent one message per line over a raw TCP connection.

K

Kernel · Ch 15 — The kernel is the core of an operating system, managing hardware, processes, and memory. Containers share the host's kernel (unlike VMs, which each run their own), which is why they are lightweight.

Keypair · Ch 40 — A **keypair** is the matched (private, public) pair generated together. The two are useless apart: the private key signs, the public key verifies, and only a matched pair agrees.

Keyword · Ch 4 — `**kwargs` collects any extra keyword arguments into a dict (`kw` = keyword). In a call, `**d` unpacks a dict into keyword arguments. Together `*args`, `**kwargs` forward an arbitrary argument list unchanged.

L

Layer · Ch 15 — A layer is the filesystem change produced by one Dockerfile instruction. Layers are cached and reused, so rebuilds skip unchanged steps; images are stacks of layers.

Lazy loading · Ch 3 — Lazy loading (lazy initialization) means deferring the creation of an expensive object until the first moment it is actually needed, rather than up front.

LEB128 · Ch 26 — LEB128 ("Little Endian Base 128") is the standard varint scheme Hatos's framework uses: the number is split into 7-bit groups, each stored in the low 7 bits of a byte, with the high bit of each byte acting as a "more bytes follow" flag. The reader stops at the first byte whose high bit is 0. It is the same scheme used by WebAssembly and the DWARF debug format.

Ledger · Ch 0 — A ledger is just a record of transactions and balances — historically an accountant's book. A distributed ledger is one copy of which is held and maintained by many computers at once, with no single authoritative master copy.

Linereceiver · Ch 41 — A `LineReceiver` is a Twisted protocol that buffers raw incoming bytes and calls your `lineReceived(line)` method once per complete line (split on a delimiter, here `\n`). It saves you from reassembling lines out of arbitrary network chunks. Twisted protocols are covered in Chapter 16.

Linter · Ch 5, Ch 20 — A linter is a tool that flags suspicious or non-conforming code (style violations, likely bugs) by static analysis. `flake8` is the linter in `hathor-core`'s toolchain; a type checker is a stricter cousin.

Lock · Ch 2 — A lock (mutex) is a mechanism ensuring only one thread enters a critical section of code at a time, used to prevent race conditions. Misused locks cause deadlocks and slowdowns.

Locking script · Ch 7 — A locking script (Bitcoin: `scriptPubKey`) is the spending condition attached to an output — a small program that must be satisfied to spend it, typically "provide a valid signature for this public key." It encodes ownership without naming an owner.

Logarithm · Ch 9 — A logarithm `log_b(x)` is the exponent to which the base `b` must be raised to obtain `x` (e.g. `log2(1024) = 10`). Logarithms compress large numbers and convert multiplication into addition. Hathor's weight is a base-2 logarithm of work.

Logging · Ch 17 — Logging is the practice of emitting timestamped records of a program's activity to a destination (console, file, service), typically with severity levels and configurable routing, so behavior can be understood after the fact.

Log level · Ch 17 — A log level is a severity tag on a log entry — commonly DEBUG, INFO, WARNING, ERROR, CRITICAL — used to filter which entries are recorded or shown, so routine detail can be separated from serious problems.

Loopingcall · Ch 34, Ch 42 — A `LoopingCall` is Twisted's periodic timer: you give it a function and an interval, and it calls that function every interval seconds on the reactor thread. Because it shares the reactor, the function must be quick — a slow one blocks the node.

LSM-tree · Ch 27 — An **LSM-tree** (log-structured merge-tree) is a storage design that never overwrites data in place. Writes are appended to an in-memory buffer (plus an on-disk log for safety); when the buffer fills it is flushed as a new immutable sorted file; background compaction later merges those files and discards superseded values. It makes writes fast and crash-safe at the cost of occasional extra read work and background CPU.

M

Make · Ch 14 — Make is a build-automation tool (originally 1976) that runs commands defined in a `Makefile`. Built to compile programs by rebuilding only out-of-date files, it is widely repurposed as a task runner for named commands.

Mempool · Ch 0, Ch 35 — The mempool ("memory pool") is the set of valid transactions a node knows about that have been seen but not yet confirmed/ordered by a block. Chapter 28.

Merkle path · Ch 31, Ch 37, Ch 39 — A Merkle path (or Merkle branch) is the short list of sibling hashes that lets you prove a single item belongs to a set summarized by one root hash, without revealing the whole set. Merged mining uses it to prove the Hathor block's hash was committed inside Bitcoin's coinbase transaction.

Metaclass · Ch 39 — A metaclass is a class whose instances are themselves classes. Defining `class Counter(Blueprint)` runs the metaclass's code, which can inspect and rewrite the class before it exists. Hathor uses this to convert a Blueprint's type annotations into storage descriptors.

Metadata · Ch 0 — Metadata here means data the node computes and stores about a vertex (its height, accumulated weight, voided status, ...) as opposed to the vertex's own fixed contents. It is recomputed and updated as the ledger evolves. Chapter 25.

Metered execution · Ch 0 — Metered execution means the contract runner counts the resources a contract uses (a kind of "gas") and stops it if it exceeds a limit, so a buggy or malicious contract cannot run forever. Chapter 39.

Miner · Ch 6 — A miner is a participant that performs proof-of-work — repeatedly hashing candidate blocks to find a valid nonce — in exchange for a reward (newly minted coins and/or fees) when it succeeds.

Miner signalling · Ch 0 — Miner signalling is a voting mechanism: miners set specific bits in the blocks they produce to indicate support for a proposed upgrade, and the rule change activates once enough support accumulates over a defined window. Chapter 38.

Mnemonic · Ch 40 — A **mnemonic** here is the ordered list of BIP39 words (12-24 of them) that encodes a wallet's seed. Anyone with the words can regenerate the entire key tree, so the words must be kept as secret as a private key.

Model · Ch 18 — A Pydantic model is a class subclassing `BaseModel` whose attributes are declared with type hints. It validates and coerces data on construction and provides serialization to/from dicts and JSON.

Module · Ch 11 — A module is a single `.py` file, importable as a namespace whose attributes are the names defined at its top level. The module's name is the filename without `.py`.

Multi-paradigm · Ch 1 — A multi-paradigm language supports several programming styles (procedural, functional, object-oriented) rather than forcing one. Python, JavaScript, and Scala are multi-paradigm; older languages often were not.

Multi-stage build · Ch 15 — A multi-stage build uses multiple `FROM` stages in one Dockerfile, doing the heavy building in early stages and copying only the needed results into a final, smaller image — leaving build tools behind.

Mypy · Ch 5, Ch 23 — `mypy` is a static type checker for Python: it reads type annotations and flags mismatches before the program runs. `hathor-core` runs it in CI (Ch 20), so untyped objects weaken its coverage.

N

Namedtuple · Ch 37, Ch 41 — A `NamedTuple` is an immutable Python record type: like a tuple, but with named fields (`template.reward` instead of `template[1]`). Once created it cannot be changed, which suits a template — it describes a fixed snapshot of work.

Namespace · Ch 11 — A namespace is a mapping from names to objects — a "where names live" container. Modules, packages, classes, and function locals are all namespaces; the dot operator reaches into them.

Node · Ch 6, Ch 8 — A node is one computer participating in the network, running the protocol software and (for a full node) keeping and verifying a complete copy of the ledger. `hathor-core` is a full-node implementation.

Nominal typing · Ch 5 — Nominal typing decides compatibility by declared name/inheritance — an object fits only if its class explicitly is (or subclasses) the required type. ABCs (§1.7) are nominal.

Nonce · Ch 6, Ch 9, Ch 37 — A nonce ("number used once") is a counter field in a block whose only purpose is to be varied during mining. Changing the nonce changes the block's hash; the miner tries nonce after nonce until the hash falls below the target. A Hathor block's nonce is 16 bytes; a transaction's is 4.

O

Observability · Ch 17 — Observability is the degree to which a system's internal state can be understood from its external outputs (logs, metrics, traces). Structured logs are a pillar of it, especially for systems no one watches live.

Observer · Ch 3 — The observer pattern lets an object (the subject) maintain a list of dependents (observers) and notify them automatically when its state changes, typically by calling a registered callback.

Oldname · Ch 33 — The `_old_` prefix and the docstring's "New method" are historical leftovers from a refactor; in the current code this private worker is the live ingestion path. The name is rot, not a clue — read it as "the core on-new-vertex worker."

Orchestrator · Ch 42 — An orchestrator is a system (Kubernetes is the common one) that runs and supervises containerized services automatically — starting them, restarting failed ones, and routing traffic only to healthy ones. It learns a service's state by calling its health-check endpoints.

Overriding · Ch 1 — Overriding is redefining, in a subclass, a method that already exists in the superclass, so instances of the subclass use the new version. Distinct from overloading (multiple methods with the same name but different parameters), which Python does not do in the classic sense.

P

Package · Ch 11 — A package is a directory of modules (and sub-packages) treated as one importable unit, traditionally marked by an `__init__.py` file. Its dotted import name mirrors its directory path.

Parallelism · Ch 2 — Parallelism is executing multiple tasks at literally the same instant, which requires multiple processors/cores. All parallel programs are concurrent; not all concurrent programs are parallel.

Parents · Ch 8 — Parents are the earlier vertices a vertex attaches to and confirms — the DAG's structural (topology) edges. Stored as a list of vertex hashes. The generalization of a blockchain's single `prev` pointer.

"Parse, don't validate" · Ch 18 — "Parse, don't validate" is a design maxim: instead of repeatedly checking untrusted data throughout the code, parse it once at the boundary into a type that cannot hold invalid data, so downstream code can trust it by construction.

Path · Ch 12 — `PATH` is an environment variable holding an ordered list of directories the shell searches to resolve a typed command name. Activating a virtual environment prepends its `bin/` so the environment's `python / pip` are found first.

Patricia trie · Ch 39 — A Patricia trie is a compressed (radix) trie: chains of single-child nodes are merged into one node holding the whole shared substring, keeping the tree shallow and lookups efficient. Hathor combines this with Merkle hashing for verifiable state.

Peer-to-peer · Ch 0 — Peer-to-peer (P2P) means every participant is an equal "peer" that connects directly to other peers, with no central server in the middle. Contrast a client-server model, where everyone talks through one company's servers.

PEP · Ch 13 — A PEP (Python Enhancement Proposal) is a design document defining a Python standard or feature. PEP 518 and PEP 517 standardized `pyproject.toml` and the build-backend interface, so packaging isn't tied to one tool.

Persistent · Ch 39 — A persistent data structure (in the data-structure sense, unrelated to disk persistence) is one where updates produce a new version while leaving every previous version intact and accessible. Hathor's trie copies a path on each write, so every past state root still names a complete snapshot — which is what makes reorgs cheap to undo.

Phony target · Ch 14 — A phony target (declared with `.PHONY`) is a Make target that does not correspond to a file, so Make runs its recipe every time rather than skipping it based on file timestamps. Essential when using Make as a task runner.

Pip · Ch 12 — pip is Python's standard package installer. It downloads packages (by default from PyPI) and installs them into the active environment's site-packages. Poetry uses the same underlying mechanisms but adds dependency resolution and locking.

Poetry · Ch 13 — Poetry is a Python dependency manager and packaging tool. It declares dependencies in `pyproject.toml`, resolves and pins them in `poetry.lock`, manages a project's virtual environment, and builds distributable packages.

Polling · Ch 36 — **Polling** is a client repeatedly asking the server "is there anything new?" on a timer, to approximate a live feed. Each poll is a full request even when the answer is "no," which makes it wasteful — most polls return nothing.

Polyfill · Ch 4 — A polyfill is code (the term comes from web development) that implements a feature the current environment lacks, so older runtimes behave like newer ones. A kind of shim for missing features.

Polymorphism · Ch 1 — Polymorphism lets a single piece of code operate on objects of different types, each responding to the same method name in its own way.

Precedence · Ch 19 — Precedence is the rule deciding which source wins when the same option is set in more than one place. `configargparse`'s order, highest first, is: command-line flag, environment variable, config file, default.

Preemptive · Ch 2 — Preemptive multitasking lets the operating system suspend a thread at any moment to run another. The opposite is cooperative, where a task keeps running until it voluntarily yields.

Privacy · Ch 1 — Python has no enforced `private`. Convention: one leading underscore (`_x`) means "internal, please don't touch"; two leading underscores (`__x`) trigger name mangling, a stronger but still bypassable form of hiding. The community relies on discipline over locks.

Private key · Ch 40 — A **private key** is a large secret number that you never reveal. Possessing it is what defines ownership; anyone who learns it can spend your coins. In Hathor it is a 256-bit `secp256k1` key, stored encrypted at rest.

Processor · Ch 17 — A processor in `structlog` is a function in the pipeline that receives and transforms a log event dict (adding a timestamp, level, bound context, etc.) before the final renderer turns it into output.

Prometheus · Ch 42 — **Prometheus** is the de-facto-standard open-source monitoring system for infrastructure. It pulls (scrapes) numeric metrics from targets on a fixed interval, stores them as time-series, and lets you query and graph them (commonly via Grafana). Covered in §42.4.

Proof-of-Authority · Ch 22, Ch 31, Ch 32 — **Proof-of-Authority** (PoA) — a consensus model for permissioned networks in which a fixed, configured set of authorized signers (identified by public key) take turns producing blocks, each block carrying a valid signature instead of a proof-of-work nonce. No mining; trust is placed in a known set of authorities.

Proof-of-work · Ch 0, Ch 6, Ch 16 — Proof-of-work (PoW) is a scheme where producing a valid block (or transaction) requires finding a number that makes its hash fall below a target — hard to do, trivial to check. The hashing is CPU-bound, which is why Hathor runs it on a thread pool. Full treatment in Ch. 9.

Protobuf · Ch 26 — Protobuf (Protocol Buffers) is Google's schema-driven binary serialization tool: you write a `.proto` schema and it generates reader/writer code. It is compact and convenient, but its output is not guaranteed to be byte-for-byte canonical across implementations, which is why it is unsuitable for data whose hash must match on every node.

Protocol · Ch 5, Ch 27, Ch 34 — A **Protocol** (Python `typing.Protocol`) is a structural interface: any object that happens to have the right methods satisfies it, with no explicit inheritance required ("if it walks like a duck..."). It lets a function accept any object shaped correctly, not just one named subclass. Full treatment in Chapter 5.

Proxy pattern · Ch 3, Ch 39 — The proxy pattern (Chapter 3) places a stand-in object between a client and a real service so the stand-in can control, restrict, or mediate access. The `BlueprintEnvironment` is a proxy: contract code talks to it, and it forwards each request to the Runner under controlled, checked conditions.

Public key · Ch 40 — A **public key** is derived from the private key and may be shared freely. It is used to verify signatures and to derive your address. Computing it from the private key is easy; the reverse is infeasible.

Pub-sub · Ch 0, Ch 3, Ch 42 — Pub-sub (publish-subscribe) is a messaging pattern: publishers announce events without knowing who listens, and subscribers register interest in event types. The metrics object subscribes to events like "new tx accepted" to update its numbers on the spot. Full treatment in Chapter 30.

Pure function · Ch 1 — A pure function always returns the same output for the same input and has no observable effect beyond that return value. `len(x)` is pure; a function that prints, or edits a global, is not.

Pydantic · Ch 18, Ch 30 — Pydantic is a Python library for declaring data models as classes with typed fields; it validates and coerces data at runtime and can serialize to/from JSON. Full treatment in Ch. 18. Here it gives `BaseEvent` its field types and the validator that keeps each event's data shape matching its type.

Pydantic v2 · Ch 18 — Pydantic v2 is the current major version, with a rewritten core and APIs including `ConfigDict`, `field_validator`, and `model_validator`. It is faster than v1 and is what `hathor-core` uses.

PyPI · Ch 12 — PyPI (the Python Package Index, at pypi.org) is the central public repository of third-party Python packages that pip downloads from by default.

Pyproject · Ch 21 — `pyproject.toml` is the standard configuration file for a modern Python project — dependencies, build settings, and tool config in one place. Hathor uses Poetry to read it. Full treatment in Chapter 13.

Pytest · Ch 20 — pytest is the most widely used Python testing framework. It discovers test functions, runs them, provides rich assertions and fixtures, and reports results. Extensible via plugins like `pytest-xdist` and `pytest-cov`.

Pytest-xdist · Ch 20 — pytest-xdist is a pytest plugin that distributes tests across multiple CPU cores (or machines), running them in parallel to shorten total test time. Enabled here with `-n auto`.

Python Virtual Machine · Ch 11 — The Python Virtual Machine (PVM) is the part of CPython that executes bytecode — a loop that reads and performs bytecode instructions one at a time. "Virtual machine" here means a software CPU, not a whole virtualized computer.

R

Race condition · Ch 2 — A race condition is a bug where the result depends on the unpredictable timing/order in which concurrent tasks run — e.g. two threads incrementing the same counter and losing an update.

Reactor · Ch 0, Ch 34, Ch 35, Ch 42 — The reactor is the heart of the Twisted framework: a single event loop that waits for events (data on a socket, a timer firing) and calls the one piece of your code registered to handle each. The whole node runs on it, on one thread. Full treatment in Chapters 2 and 16.

Reactortype · Ch 23 — Twisted predates Python's modern type system, so its reactor is exported as a plain object satisfying many interfaces rather than as one statically-typed class. Without help, a checker treats most accesses on it as untyped.

Read-only · Ch 22 — A read-only object is one that nothing is supposed to mutate after it is created; configuration is treated this way so no stray code can change a network constant mid-run. (Pydantic models are mutable by default unless explicitly frozen; in Hathor the read-only discipline is enforced partly by convention and partly by the load-time `extra='forbid'` validation that rejects malformed input in the first place.)

Re-entrancy · Ch 39 — Re-entrancy is when a contract, mid-call, calls back into itself (often via another contract) before the first call finished. It is a classic source of exploits because the contract's state may be half-updated. Hathor forbids it by default and a method must explicitly opt in with `allow_reentrancy=True`.

Reorg · Ch 6, Ch 10, Ch 25, Ch 32, Ch 35, Ch 39, Ch 43 — **Reorg** (reorganization) — when a node abandons part of its current best chain of blocks in favor of a competing chain that has accumulated more work (a higher score). Blocks on the abandoned branch become voided; their transactions return to the mempool or are re-confirmed by the new chain. The reorg size is how many blocks deep the switch goes.

REPL · Ch 11 — The REPL (Read-Eval-Print Loop) is the interactive Python prompt: it reads a line, evaluates it, prints the result, and loops. Started by running `python` with no arguments.

Request-response · Ch 36 — **Request-response** is the interaction model of plain HTTP: the client sends a request and the server sends back exactly one response, then the exchange is finished. The server cannot initiate; it can only reply.

RocksDB · Ch 0 — **RocksDB** is an embedded key-value database: a fast on-disk store of `key → value` byte pairs that runs inside the node's own process rather than as a separate server. Full treatment, and the comparison against alternatives like MongoDB, in Chapter 27.

S

Secp256k1 · Ch 40 — **secp256k1** is the name of the specific elliptic curve — a fixed, public set of parameters — that Hathor, Bitcoin, and Ethereum all use. Because it is a shared standard, keys and signatures are interoperable across tools.

Seed · Ch 40 — A **seed** is the master secret (a block of bytes) from which an HD wallet derives all its keys. It is produced from the BIP39 mnemonic (plus an optional passphrase) and fed into BIP32 derivation.

Semantic versioning · Ch 13 — Semantic versioning (semver) is a convention where a version `MAJOR.MINOR.PATCH` signals the kind of change: PATCH = bug fixes, MINOR = backward-compatible features, MAJOR = breaking changes. Constraint operators like `^` rely on this promise.

Sentry · Ch 17 — Sentry is an error-tracking service that aggregates and alerts on exceptions from running software. structlog can forward error events to it via a processor; it is an optional dependency in `hathor-core`.

Sequence number · Ch 39 — A sequence number (seqnum) is a per-caller counter that must strictly increase with each call. It prevents replay: an attacker cannot resubmit a previously signed transaction, because its seqnum is no longer higher than the last one the node accepted.

Server push · Ch 36 — **Server push** means the server sends data to the client without the client having requested that specific data, using a connection kept open for the purpose. It is the inverse of polling.

Server-Sent Events (SSE) · Ch 36 — **Server-Sent Events (SSE)** is a simpler push standard where the server holds an HTTP response open and writes events to it over time. It is one-directional (server→client only), which is why it cannot serve a stream where the client must also subscribe, ack, and adjust flow control.

Shim · Ch 4 — A shim is a small compatibility layer that transparently intercepts calls and bridges a difference (between API versions, platforms, or libraries), letting surrounding code stay unaware of the difference.

Side chain · Ch 32 — **Side chain** — a branch of blocks whose score is lower than the current best chain's. Its head (and the blocks below it that are off the best chain) are voided. A side chain becomes the best chain — triggering a reorg — only if it later overtakes the current best chain's score.

Side effect · Ch 1 — A side effect is any change a function makes to the world outside itself — modifying a global variable, writing a file, printing, mutating an argument. Pure functional code minimizes side effects.

Sighash · Ch 25, Ch 40 — The sighash ("signature hash") is the hash of a transaction's body that a signer actually signs. Each input's `data` (unlocking script) is blanked while computing it, because you cannot sign over the signature you are in the middle of producing. `Transaction` caches it (`transaction.py:93`) because signing many inputs re-serializes the same body repeatedly.

Signal · Ch 41 — A signal is a small asynchronous notification the operating system delivers to a process (e.g. `SIGINT` from Ctrl-C). `SIGUSR1` and `SIGUSR2` are left undefined by the OS for an application to use as it likes. A signal handler can interrupt the program at almost any point, which is why `sysctl` restricts what may run from the `SIGUSR2` path.

Singledispatch · Ch 4 — `functools.singledispatch` is a Python standard-library decorator that turns a function into one whose implementation is chosen by the type of its first argument, with implementations registered via `@fn.register`.

Singleton · Ch 3, Ch 23 — A singleton is a resource of which exactly one instance exists per process. An event loop is inherently one-per-process, which makes the reactor a natural singleton. Full treatment in Ch. 3.

Site-packages · Ch 12 — `site-packages` is the directory where third-party (pip-installed) packages are placed. There is a global one for the system interpreter and a private one inside each virtual environment; which is used depends on which `python` runs.

Slots · Ch 25 — `__slots__` declares the fixed set of attributes instances may have, replacing each object's per-instance dictionary with a compact fixed layout. It saves memory (decisive when millions of vertices are in RAM) and prevents accidental attribute typos. It is a Python optimization, not a domain concept.

Smart contract · Ch 0, Ch 39 — A smart contract is a program stored on a ledger whose code is executed and agreed upon by every node in the network, so it can hold and move tokens according to its own public rules with no trusted operator. Hather's variant is called a nano-contract.

Stack · Ch 31 — A stack is a last-in-first-out collection: you can only add (push) to the top and remove (pop) from the top. Think of a stack of plates. A stack machine evaluates a program using only one such stack as its working memory.

State · Ch 1 — State is the data an object currently holds — its attribute values at a moment in time. An object with state "remembers" things between method calls; that memory is what makes it stateful.

State machine · Ch 34 — A state machine (finite-state machine) is a model in which an object is always in exactly one of a fixed set of states, certain events trigger transitions between states, and each state accepts only a limited set of events. It makes a connection's lifecycle explicit and rejects out-of-phase messages by construction.

State root · Ch 39 — A state root is the Merkle root of a state store — one hash that uniquely fingerprints all of the state. Two parties can confirm their states are identical by comparing this one value. Hathor keeps a per-contract state root and a per-block root over all contracts.

Static · Ch 5 — Static (in "static typing"/"static analysis") means "determined from the source ahead of run time," without executing the program. Its opposite is dynamic — determined while running.

Static analysis · Ch 5 — Static analysis is examining a program's source to learn facts about it (type errors, unused variables, security issues) without running it. Type-checking and linting are forms of static analysis.

Static metadata · Ch 38 — Static metadata is per-vertex data the node computes once and then never changes — as opposed to mutable metadata that gets updated as the ledger evolves (the split is covered in Ch. 25). A block's height and its `feature_activation_bit_counts` are static: once a block is fixed in the chain, both are fixed too, so they can be computed once and cached.

Staticmethod · Ch 4 — `@staticmethod` marks a method that receives neither `self` nor the class — a plain function namespaced inside a class. Use it for helpers that logically belong to the class but need no instance.

Strenum · Ch 38 — A `StrEnum` (Python 3.11+) is an enumeration whose members are `str` values. `Feature.NANO_CONTRACTS == 'NANO_CONTRACTS'` is `True`, and `Feature.NANO_CONTRACTS.value` is the string. This lets the settings file refer to features by plain string names while the code uses the type-safe enum.

Structural typing · Ch 5, Ch 16, Ch 23 — Python's `typing.Protocol` enables structural typing: any object that has the required methods/attributes satisfies the protocol, with no need to inherit from it. It is a static-typing tool, checked by `mypy`, and is unrelated to Twisted's connection-handler `Protocol` class. Full treatment in Ch. 5.

Structured logging · Ch 0, Ch 17, Ch 21 — Structured logging records log entries as machine-readable key-value data (e.g. `event="block accepted" height=42`) rather than as freeform sentences, so the logs can be searched and analyzed programmatically. Hathor uses a library called `structlog`; full treatment in Chapter 17.

Subsystem · Ch 3 — A subsystem is a cohesive group of classes/modules that together provide some capability (e.g. "storage" or "networking"). A facade fronts a subsystem.

Sybil attack · Ch 6 — A Sybil attack is the creation of many fake identities by one actor to gain disproportionate influence in a system that counts participants. Open, permissionless networks must make influence cost a scarce resource to resist it. (Named after a case study of a person with many identities.)

Symbolic link · Ch 12 — A symbolic link (symlink) is a filesystem entry that points to another file or directory — like a shortcut. A virtual environment's `python` is typically a symlink to a real interpreter, so the environment needn't copy all of Python.

Syncing · Ch 0 — Syncing (synchronizing) is the process by which a node that is behind downloads the blocks and transactions it is missing from its peers until its copy of the ledger matches the network's.

Sync-v2 · Ch 0 — Sync-v2 is the current peer-to-peer synchronization protocol. An earlier sync-v1 existed but has been removed; only sync-v2 remains. Chapter 35.

Syntactic sugar · Ch 4 — Syntactic sugar is syntax that makes something easier to write or read but adds no new capability — it could always be expressed the longer way. `@decorator` is sugar for `f = decorator(f)`.

Sysmodules · Ch 11 — `sys.modules` is a dictionary CPython keeps of every module already imported in the current process, keyed by dotted name. Imports check it first, which is why a module's top-level code runs only once.

Syspath · Ch 11 — `sys.path` is the ordered list of directories Python searches to find a module on import. It includes the standard library and the active environment's installed-packages directory; the first match wins.

T

Target · Ch 6, Ch 9 — A target is the numeric threshold a vertex's hash must fall below to be valid proof-of-work. In Hather it is derived from the vertex's weight: roughly $2^{(256 - \text{weight})}$, so higher weight means a smaller target and more work.

Task runner · Ch 14 — A task runner is a tool that defines and runs a project's common named commands (test, lint, build) from one place, so contributors and CI invoke identical commands without memorizing them.

TCP · Ch 36 — **TCP** (Transmission Control Protocol) is the reliable, ordered byte-stream transport that both HTTP and WebSockets run on top of. A WebSocket reuses the very TCP connection that carried the initial HTTP request — the upgrade does not open a new socket.

Test runner · Ch 20 — A test runner discovers and executes a project's automated tests and reports results. `pytest` is the runner here; it finds test functions, runs them, and summarizes passes and failures.

Thread · Ch 2 — A thread is an independent sequence of execution within a process. Multiple threads in one process share the same memory, which makes communication easy and corruption easy.

Thread pool · Ch 2, Ch 16 — A thread pool is a managed set of reusable worker threads. Jobs are submitted to the pool rather than spawning a new thread each time; used to run blocking or CPU-bound work off the main event loop. Full treatment in Ch. 2.

Tip · Ch 28 — A **tip** of the ledger DAG is a vertex that no other vertex confirms yet — a leaf at the growing frontier of the graph. A **mempool tip** is such a tip that is also still unconfirmed by any block. New transactions attach to (confirm) existing tips, so the node must always know what its current tips are. Full treatment of the DAG and tips in Chapter 8.

Tips · Ch 8 — Tips are the vertices at the frontier of the DAG that no other vertex has yet confirmed (named as a parent). New vertices select their parents from among the tips.

Topological order · Ch 8, Ch 35 — A topological order of a directed acyclic graph is a linear ordering of its vertices such that every vertex comes after all the vertices it depends on (points to). Processing a DAG in topological order guarantees you never reach an item before its prerequisites. Full treatment of the DAG in Ch. 8.

Transitive · Ch 13 — A transitive dependency is a dependency of a dependency — a package you don't ask for directly but that something you depend on needs. The full set forms the transitive dependency tree.

Trie · Ch 39 — A trie (or prefix tree) stores keys by spelling them out along a path from the root, so keys with a shared prefix share the top of their path. A radix/Patricia trie is the compressed form that collapses single-child chains into one node.

Twelve-factor · Ch 19 — Twelve-factor refers to a set of widely-cited principles for building deployable web/network software. One factor is that configuration should come from the environment, keeping config separate from code.

Twistederror · Ch 23 — Twisted's own `ReactorAlreadyInstalledError` is raised by `asyncioreactor.install` when a reactor is already in place. Hathor catches it only inside the `asyncio` branch, to replace the bare error with a message explaining the likely cause (an indirect default-reactor install at import time).

Type annotation · Ch 5 — A type annotation (type hint) is syntax attaching an expected type to a variable, parameter, or return value (`x: int`, `def f() -> str`). In Python it is informational by default — read by tools and humans, not enforced by the interpreter.

Type variable · Ch 5 — A type variable (`TypeVar`) is a placeholder standing for "some specific type, the same throughout this signature," used to write generics.

`def first(x: list[T]) -> T` returns the list's element type.

U

Unlocking script · Ch 7 — An unlocking script (Bitcoin: `scriptSig`) is the data an input supplies to satisfy the locking script of the output it spends — usually a signature and public key. In Hathor it is the input's `data` field.

UTXO model · Ch 7 — The UTXO model (Unspent Transaction Output) records ownership as a set of discrete outputs created by past transactions and not yet spent. A balance is the sum of one's unspent outputs. Used by Bitcoin and Hathor.

V

Validator · Ch 22 — A validator (in Pydantic) is a method that runs while the model is being built, to coerce raw input into the declared type (`mode='before'`) or to check relationships between fields once they are all set (`mode='after'`). It is how a human-friendly YAML shape becomes a strongly-typed, internally-consistent object — or is rejected.

Varint · Ch 26 — A varint (variable-length integer) is an integer encoding that uses fewer bytes for smaller numbers — one byte for small values, more only as needed — instead of a fixed width. It saves space when most values are small.

Verifyobject · Ch 23 — `zope.interface.verify.verifyObject(Interface, obj)` checks at runtime that `obj` actually provides every method the interface declares, raising if not. Hathor uses it to confirm the real reactor matches `ReactorProtocol` before casting to that type.

Vertex · Ch 0, Ch 8, Ch 33 — A vertex in `hathor-core` is any node of the ledger DAG — a block or a transaction. The umbrella term used when the kind does not matter; implemented as `GenericVertex` (aliased `BaseTransaction`).

Vertexid · Ch 25 — `VertexId` is a type alias for `bytes` (`hathor/types.py:26`) — specifically the 32-byte double-SHA256 hash that identifies a vertex. Storing parents and inputs as `VertexId` rather than as object references keeps a vertex small and lets the storage layer load the graph lazily.

Virtual environment · Ch 12 — A virtual environment is an isolated, self-contained Python setup for one project — a directory with its own interpreter link and its own package directory — so each project's dependencies stay separate from other projects' and from the system Python.

Virtual machine · Ch 15 — A virtual machine emulates a complete computer, running a full guest operating system on a hypervisor. Stronger isolation than a container but far heavier, because each VM ships and boots an entire OS.

Voided · Ch 33, Ch 35 — A vertex is voided when consensus marks it as not part of canonical history (e.g. it was on a chain that lost a reorg, or it conflicts with a heavier transaction). Voided does not mean deleted — it means "recorded but not counted." Ch. 10 & 32.

Voiding · Ch 10 — Voiding marks a vertex as not counted toward the real ledger (its outputs unspendable, its effects undone) without deleting it, via the `voided_by` metadata field. Reversible if consensus later changes.

W

WebSocket · Ch 30 — A WebSocket is a persistent, two-way connection between a client and a server over a single long-lived TCP link, unlike HTTP's request-then-close model. It lets the node push events to a client the moment they happen, rather than the client polling. Full treatment in Ch. 36.

Weight · Ch 0, Ch 22, Ch 32, Ch 33 — **Weight** — a number measuring how much proof-of-work a vertex represents ($\text{weight} = \log_2$ of the expected number of hash attempts). **Accumulated weight** sums a vertex's weight with all the work piled up behind it in the DAG. Consensus prefers the history with the most accumulated weight. Full treatment in Ch. 9.

Wheel · Ch 13 — A wheel (`.whl`) is Python's standard pre-built package format — an archive that installs without a compile/build step, making installation fast and deterministic. An sdist is the source-archive counterpart.

Wrapper · Ch 3 — A wrapper is any object that encloses another and forwards calls to it, usually adding or altering behavior. Adapters, proxies, and decorators are all kinds of wrapper (decorators are Chapter 4).

X

Xpub · Ch 36 — An **xpub** (extended public key) is a single public key from which a whole sequence of wallet addresses can be derived without exposing any private key. The history streamer can walk an xpub's derived addresses to stream a wallet's entire history.